

DOCUMENTATION TECHNIQUE

Moteur de Detection d'Anomalies Budgetaires

Detection d'Anomalies et Previsions

Document de reference complet pour les equipes techniques,
testeurs, clients et integrateurs ERP

VERSION

DATE

CLASSIFICATION



Table des matieres

1. Vue d'ensemble du systeme

2. Architecture technique detaillee

2.1 Stack technique

2.2 Diagramme d'architecture

2.3 Composants cles

3. Modele de donnees — schema physique

3.1 Table ai_tenants

3.2 Table ai_erp_connectors

3.3 Table ai_tenant_erp_connections

3.4 Table ai_pipelines

3.5 Table ai_invoices

3.6 Table ai_series

3.7 Autres tables

4. Authentification et gestion des roles

5. Feature A — Organisation (tenants)

6. Feature B — Connecteurs ERP

7. Feature C — Pipelines d'importation

7.1 Creation et configuration

7.2 Jointures SQL et extraction securisee

7.3 Normalisation et validation

7.4 Gestion des statuts

7.5 Saisonnalite automatique

7.6 Scoring et creation d'alertes

7.7 Detection des documents manquants

8. Feature D — Connexion tenant/ERP

9. Feature E — Alertes et feedback

10. Feature F — Budget et previsions

10.1 Philosophie : aucune persistance

10.2 Budget implicite

10.3 Simulation budgétaire

10.4 DTOs de reponse

10.5 Alertes budgetaires

10.6 Endpoints

10.7 Flux de donnees

10.8 Points clés de conception

11. Feature G — Supervision

12. Securite et isolation des donnees

13. Exemples complets de flux API

14. Codes d'erreur

15. Annexes

1. Vue d'ensemble du systeme

Logique metier

Le moteur est une application qui se connecte aux systemes de gestion d'entreprise (ERP) pour importer automatiquement les factures, bons de commande ou bons d'achat. Il analyse ensuite ces documents pour detecter des anomalies (montants suspects, doublons, factures manquantes) et aide a piloter le budget (budget implicite base sur l'historique, simulations, previsions sur 24 mois). L'accès peut se faire soit par un login classique (administrateur du moteur ou de l'organisation), soit par une integration directe dans l'ERP via une iframe, sans mot de passe supplementaire.

Logique technique

L'application est un backend Java Spring Boot exposant une API REST JSON. Elle utilise PostgreSQL comme base de donnees, Spring Security pour l'authentification JWT, et des scripts Python pour le calcul statistique (scoring, saisonnalite). L'integration avec les ERP se fait via des pipelines configurables (JDBC, CSV, API). L'isolation des donnees est stricte entre les organisations (tenants) et meme entre les differents connecteurs ERP d'une meme organisation.

2. Architecture technique detaillee

2.1 Stack technique

Tableau 1 : Stack technique complete

COUCHE	TECHNOLOGIE	VERSION	ROLE
Langage	Java	JDK 25	Logique metier
Framework	Spring Boot	4.x	Conteneur, injection de dependances
Securite	Spring Security	7.x	Authentification, autorisation
Persistence	Spring Data JPA	4.x	Acces base de donnees
Base de donnees	PostgreSQL	16+	Stockage principal
Migrations	Flyway	10.x	Gestion du schema
Chiffrement	JCA (AES-GCM)	—	Chiffrement des mots de passe DB
JWT	Nimbus JOSE + JWT	10.x	Jetons internes
Python	CPython	3.11+	Scripts de scoring et de serie
Build	Maven	3.9+	Compilation

2.2 Diagramme d'architecture

Le systeme s'articule autour de trois couches principales : les clients (interfaces utilisateur et ERP), le moteur (backend Spring Boot), et les sources de donnees (bases ERP, fichiers CSV, API externes).

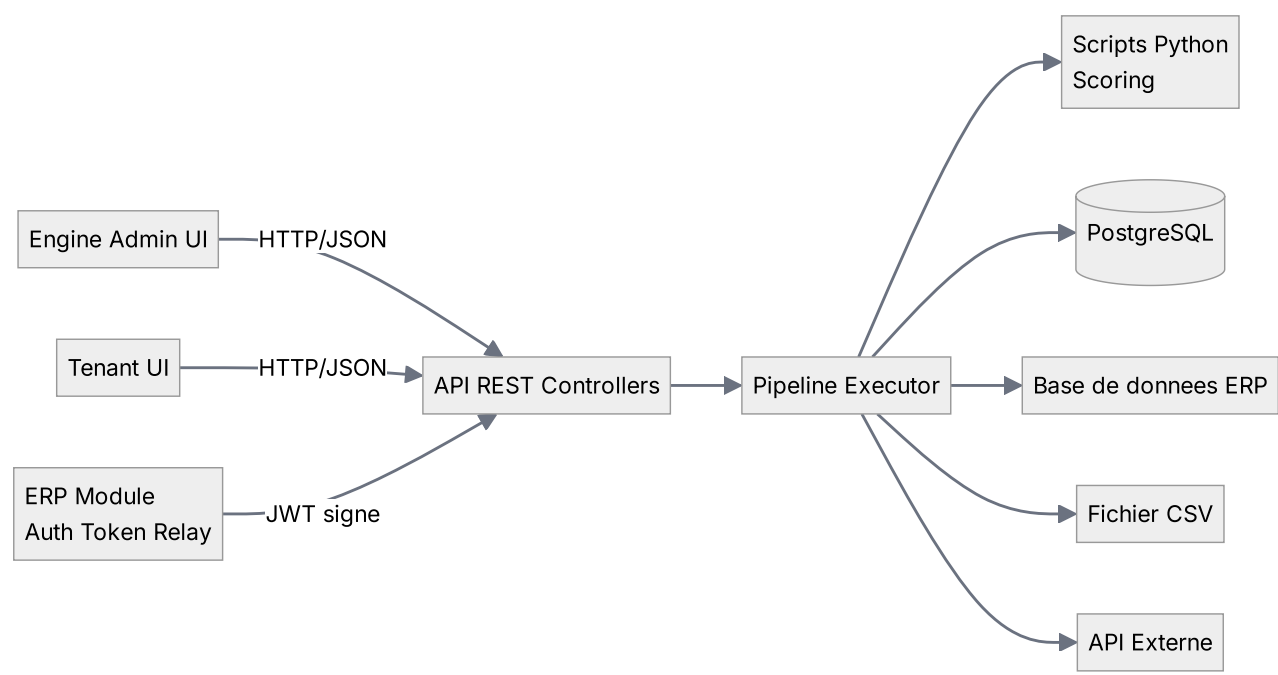


Figure 1 : Architecture globale du systeme

Tableau 2 : Flux de donnees entre composants

SOURCE	DESTINATION	PROTOCOLE / FORMAT
Engine Admin UI	API REST Controllers	HTTP / JSON
Tenant UI	API REST Controllers	HTTP / JSON
ERP (module intermediaire)	API REST Controllers	Authentification / Token Relay JWT signe
API REST	Pipeline Executor	Interne
Pipeline Executor	Scripts Python (scoring)	Process / JSON
Pipeline Executor	PostgreSQL	JDBC / SQL
Pipeline Executor	Base de donnees ERP	JDBC
Pipeline Executor	Fichier CSV	Lecture fichier
Pipeline Executor	API externe	HTTP / REST

2.3 Composants cles — roles resumes

Tableau 3 : Description des composants principaux

COMPOSANT	ROLE
Engine Admin	Administrateur du moteur (creation des tenants, connecteurs, supervision)

Utilisateur ERP	Personne accedant via l'integration iframe (aucun compte stocke)
Pipeline	Configuration d'import (JDBC, CSV, API)
Serie	Regroupement de factures par fournisseur + libelle + champs supplementaires
Scoring Python	Scripts <code>detect.py</code> (anomalies) et <code>compute_series.py</code> (saisonnalite)

3. Modele de donnees — schema physique

Pour les developpeurs — Les tables principales sont decrites ci-dessous. Les clees etrangeres utilisent `ON DELETE CASCADE` pour garantir l'integrite referentielle lors de la suppression d'un tenant.

3.1 Table `ai_tenants`

Tableau 4 : Schema de la table `ai_tenants`

COLONNE	TYPE	CONTRAINTE	DESCRIPTION
<code>id</code>	UUID	PK	Identifiant unique du tenant
<code>username</code>	VARCHAR(100)	UNIQUE, NOT NULL	Nom d'utilisateur pour l'authentification
<code>password_hash</code>	VARCHAR(255)	—	Hash BCrypt du mot de passe
<code>role</code>	VARCHAR(20)	NOT NULL	<code>ADMIN</code> ou <code>TENANT</code>
<code>name</code>	VARCHAR(255)	—	Nom d'affichage (optionnel)
<code>db_mode</code>	VARCHAR(20)	NOT NULL	<code>SHARED</code> ou <code>ISOLATED</code>
<code>db_url</code> , <code>db_username</code> , <code>db_password</code>	TEXT	—	Connexion base dediee (chiffree)
<code>created_at</code>	TIMESTAMP	NOT NULL	Date de creation

3.2 Table `ai_erp_connectors`

Tableau 5 : Schema de la table `ai_erp_connectors`

COLONNE	TYPE	CONTRAINTE	DESCRIPTION
<code>id</code>	UUID	PK	Identifiant unique
<code>name</code>	VARCHAR(100)	UNIQUE	Nom du connecteur (ex: Ask&Go)
<code>auth_type</code>	VARCHAR(50)	—	<code>JWT_SIGNED</code> (pour l'instant)
<code>public_key</code>	TEXT	—	Cle publique PEM pour validation des jetons
<code>created_at</code>	TIMESTAMP	—	Date de creation

3.3 Table `ai_tenant_erp_connections`

Tableau 6 : Schema de la table `ai_tenant_erp_connections`

COLONNE	TYPE	CONTRAINTE	DESCRIPTION
<code>id</code>	UUID	PK	Identifiant unique
<code>tenant_id</code>	UUID	FK → <code>ai_tenants(id)</code> ON DELETE CASCADE	Tenant lie
<code>connector_id</code>	UUID	FK → <code>ai_erp_connectors(id)</code> ON DELETE RESTRICT	Connecteur ERP
<code>external_id</code>	TEXT	NOT NULL, UNIQUE (<code>connector_id</code> , <code>external_id</code>)	Identifiant externe (ex: <code>whitecape_ask</code>)
<code>created_at</code>	TIMESTAMP	NOT NULL	Date de creation

3.4 Table `ai_pipelines`

Tableau 7 : Schema de la table `ai_pipelines`

COLONNE	TYPE	DESCRIPTION
<code>id</code>	UUID	PK
<code>tenant_id</code>	UUID	FK → <code>ai_tenants(id)</code> ON DELETE CASCADE
<code>name</code>	VARCHAR(255)	Nom du pipeline
<code>source_type</code>	VARCHAR(20)	<code>JDBC</code> , <code>CSV</code> ou <code>API</code>
<code>status</code>	VARCHAR(20)	<code>DRAFT</code> , <code>ACTIVE</code> , <code>PAUSED</code>
<code>config_json</code>	JSONB	Configuration complete (mapping, jointures, statuts, conditions, planification)
<code>connector_id</code>	UUID	FK → <code>ai_erp_connectors(id)</code> (optionnel)
<code>external_id</code>	TEXT	(optionnel) Herite de la liaison tenant-ERP
<code>active</code>	BOOLEAN	Par default true
<code>created_at</code> , <code>updated_at</code>	TIMESTAMP	Horodatage

3.5 Table `ai_invoices`

Tableau 8 : Schema de la table `ai_invoices`

COLONNE	TYPE	DESCRIPTION
<code>id</code>	UUID	PK
<code>tenant_id</code>	UUID	→ tenant
<code>pipeline_id</code>	UUID	→ pipeline
<code>connector_id</code>	UUID	→ connecteur (pour filtrage ERP)
<code>external_id</code>	TEXT	Identifiant externe
<code>series_id</code>	UUID	FK → <code>ai_series(id)</code>
<code>supplier</code>	VARCHAR(255)	Fournisseur normalise
<code>label</code>	VARCHAR(255)	Libelle (optionnel)
<code>amount</code>	NUMERIC(15,2)	Montant
<code>invoice_date</code>	DATE	Date de la facture
<code>is_final</code>	BOOLEAN	false = provisionnel, true = definitif
<code>accounting_status</code>	VARCHAR(50)	<code>RECEIVED</code> ou <code>VALIDATED</code>
<code>received_date</code>	TIMESTAMP	Date d'import du provisionnel
<code>validation_date</code>	TIMESTAMP	Date de passage en definitif
<code>status</code>	VARCHAR(20)	<code>ACTIVE</code> , <code>ANOMALY</code> , <code>TREATED</code> , etc.
<code>source</code>	VARCHAR(50)	Provenance
<code>ignored</code>	BOOLEAN	true si exclue des calculs
<code>extra_data</code>	JSONB	Champs supplementaires

3.6 Table `ai_series`

Tableau 9 : Schema de la table `ai_series`

COLONNE	TYPE	DESCRIPTION
<code>id</code>	UUID	PK
<code>tenant_id</code>	UUID	→ tenant

<code>pipeline_id</code>	UUID	→ pipeline
<code>connector_id</code>	UUID	→ connecteur (filtrage ERP)
<code>supplier</code>	VARCHAR(255)	Fournisseur
<code>label</code>	VARCHAR(255)	Libelle
<code>group_cols</code>	JSONB	Colonnes de regroupement
<code>group_key_extras</code>	JSONB	Valeurs des champs supplementaires
<code>n</code>	INTEGER	Nombre de factures definitives
<code>mu</code>	NUMERIC(15,2)	Moyenne generale
<code>sigma</code>	NUMERIC(15,2)	Ecart type
<code>cv</code>	NUMERIC(10,4)	Coefficient de variation
<code>median_gap_days</code>	INTEGER	Ecart median en jours entre dates
<code>last_date</code>	DATE	Derniere date (tous statuts)
<code>monthly_mu</code>	JSONB	Map mois → moyenne historique
<code>use_seasonality</code>	BOOLEAN	Detecte automatiquement par Python
<code>tolerance_pct</code>	NUMERIC(5,2)	Tolerance en pourcentage (defaut 10)
<code>tolerance_days</code>	INTEGER	Tolerance en jours (defaut 10)
<code>forecast_start_today</code>	BOOLEAN	Si true, les previsions commencent aujourd'hui
<code>active</code>	BOOLEAN	Serie active

3.7 Autres tables

- `ai_anomalies` : stocke les scores calcules par Python (une par facture et par serie).
- `ai_alerts` : alertes creees lorsqu'un score > 60.
- `ai_feedback` : decisions des utilisateurs sur les alertes (confirmer/rejeter/ignorer) avec anciennes/nouvelles tolerances.
- `ai_budgets` : table non utilisee (budgets explicites non persistes), mais conservee pour compatibilite.
- `ai_pipeline_runs` , `ai_pipeline_run_logs` : traces d'execution des pipelines.

4. Authentification et gestion des roles

Logique metier

Deux types d'utilisateurs peuvent se connecter directement par login/mot de passe :

- **Administrateur du moteur (Engine Admin)** : cree les organisations clientes et les connecteurs ERP, supervise le systeme.
- **Administrateur d'une organisation (Tenant)** : configure les pipelines, gere les alertes et le budget.

Les utilisateurs de l'ERP (ex: commerciaux) n'ont pas de compte dans le moteur ; ils accedent via une integration iframe (voir **Feature D**). Leurs droits sont limites a la consultation des alertes et au feedback.

Tableau 10 : Matrice des roles et endpoints accessibles

ROLE	ENDPOINTS ACCESSIBLES
ADMIN	<code>/admin/**</code> (tenants, connecteurs, supervision)
TENANT	<code>/pipelines/**</code> , <code>/invoices/**</code> , <code>/alerts/**</code> , <code>/feedback/**</code> , <code>/series/**</code> , <code>/budget/**</code> , <code>/tenants/{tenantId}/erp-connections</code> (son propre tenant)

Les utilisateurs ERP (sans role Spring) accedent aux memes endpoints que **TENANT** mais leurs donnees sont filtrees par `connector_id` et `external_id`.

Logique technique

Flux technique — `POST /auth/login` avec `{ "username": "...", "password": "..." }` → recherche le tenant par `username` dans `ai_tenants`, verifie le mot de passe (BCrypt), genere un JWT signe avec `JWT_SECRET` (expiration 1 heure) contenant `tenant_id`, `username` et `roles` (`["ADMIN"]` ou `["TENANT"]`).

5. Feature A — Organisation (tenants)

Logique metier

L'administrateur du moteur cree une fiche pour chaque organisation cliente (ex: Whitecape). Il choisit un nom d'utilisateur unique (ex: `whitecape_admin`) et un mot de passe (au moins 8 caracteres, avec majuscule, minuscule, chiffre, caractere special). Le systeme stocke ce compte et lui attribue le role `TENANT`. L'administrateur tenant peut ensuite se connecter avec ce compte pour configurer ses propres pipelines.

Logique technique

Endpoint : `POST /admin/tenants`

Corps :

```
{ "username": "whitecape_admin", "password": "Secure123!", "name": "Whitecape" }
```

Validations : unicite du `username`, longueur du mot de passe ≥ 8 , complexite (regex). Le mot de passe est hashe avec BCrypt.

Reponse (201) :

```
{ "id": "...", "username": "...", "name": "...", "role": "TENANT" }
```

Liste des tenants : `GET /admin/tenants` (page) retourne `{ id, username, name, createdAt, role }`.

Suppression : `DELETE /admin/tenants/{id}` → suppression physique en cascade de toutes les donnees liees. En mode `ISOLATED`, la base dediee est egalement supprimee.

6. Feature B — Connecteurs ERP

Logique metier

Avant qu'une organisation puisse intégrer son ERP, l'administrateur moteur doit déclarer le type d'ERP (ex: Ask&Go) comme "connecteur disponible". Il lui donne un nom et y associe la cle publique de l'ERP (format PEM). Cette cle servira a valider les jetons d'authentification que l'ERP enverra plus tard. Un connecteur ne peut pas etre supprime s'il est encore utilise par une organisation.

Logique technique

Creation : `POST /admin/connectors` — `{ "name": "Ask&Go", "authType": "JWT_SIGNED", "publicKey": "-----BEGIN PUBLIC KEY-----..." }`

Liste : `GET /admin/connectors` — retourne tous les connecteurs avec la liste des tenants lies.

Suppression : `DELETE /admin/connectors/{id}` — si des `TenantErpConnection` existent, renvoie **409 Conflict** avec la liste des tenants concernes.

NOTE

Le diagramme de sequence de suppression d'un connecteur ERP n'est pas disponible en raison de contraintes de format. La logique est decrite ci-dessus : verification des dependances `TenantErpConnection`, puis suppression ou retour d'erreur 409.

7. Feature C — Pipelines d'importation

Logique metier

Un pipeline est un "plan" qui indique au moteur comment aller chercher les factures dans l'ERP. L'administrateur tenant le cree, parametre la connexion (base de donnees, fichier CSV ou API), definit les jointures entre tables, associe les colonnes aux champs standard (fournisseur, montant, date, libelle), et precise quels statuts correspondent a des documents provisionnels (ex: RECU) ou definitifs (ex: COMPTABILISE). Une fois configure, il peut executer le pipeline manuellement ou le planifier.

7.1 Creation et configuration

Tableau 11 : Etapes de creation et configuration d'un pipeline

ETAPE	ENDPOINT	DESCRIPTION
1. Creation	POST <code>/pipelines</code>	Avec <code>{ "name": "Factures Ask&Go", "sourceType": "JDBC" }</code> → retourne <code>{ "id": "pipeline-jdbc" }</code> , statut DRAFT .
2. Configuration (preview)	POST <code>/pipelines/{id}/preview</code>	Avec un objet <code>PipelineConfigDTO</code> contenant les parametres specifiques au type de source.
3. Sauvegarde	PUT <code>/pipelines/{id}/mapping</code>	Enregistre la configuration dans <code>config_json</code> .

Logique technique — Parametres de configuration

JDBC : `jdbcUrl`, `jdbcUsername`, `jdbcPassword`, `tables` (nom, alias), `joins` (type, fromAlias, toAlias, condition), `tenantIdColumn` (pour filtrage automatique).

CSV : upload du fichier via `multipart/form-data`.

API : `apiEndpoint`, `authHeader`, `authToken`.

Commun : `importStatusColumn`, `importStatuses`, `provisionalStatuses`, `finalStatuses`, `fieldMapping` (supplier, amount, date, label, extraFields, groupCols), `schedule` (mode, cron/interval), `conditions` (liste structuree).

Le systeme execute la requete (ou lit le CSV) et retourne les colonnes + un echantillon de 5 lignes. Si `supplier`, `amount` et `date` sont mappes, le pipeline est automatiquement execute une premiere fois.

7.2 Jointures SQL et extraction securisee

Le `JdbcAdapter` construit la requete SQL a partir des `tables`, `joins` et `conditions`. Les conditions sont transformees en parametres (`?`) et executees avec `PreparedStatement`, eliminant tout risque d'injection SQL.

EXEMPLE DE REQUETE GENEREE

```
SELECT fr.nom AS supplier,
       f.montant AS amount,
       f.date_facture AS invoice_date,
       f.libelle AS label
FROM factures f
INNER JOIN fournisseurs fr ON f.fournisseur_code = fr.code
WHERE f.tenant_code = ? AND f.amount > ?
```

7.3 Normalisation et validation des donnees

`InvoiceNormalizer` transforme chaque ligne en `InvoiceDTO` avec les regles suivantes :

- **Fournisseur** : obligatoire, non vide.
- **Montant** : numerique, > 0 ; supporte `1,000` (mille) et `1,50` (virgule decimale).
- **Date** : accepte plusieurs formats (`yyyy-MM-dd`, `dd/MM/yyyy`, etc.) ; echoue sinon.
- **Statut** : compare a `importStatuses` ; si non present, la ligne est ignoree.

Les lignes invalides sont loggees dans `ai_pipeline_run_logs` avec `errorsJson` (numero de ligne, donnees brutes, raison).

7.4 Gestion des statuts — flux RECU → COMPTABILISE

Tableau 12 : Mapping des statuts comptables

STATUT SOURCE	ISFINA L	ACCOUNTINGSTA TUS	REMARQUE
RECU (provisionnel)	false	RECEIVED	N'entre pas dans les moyennes historiques
COMPTABILISE (definitif)	true	VALIDATED	Devient la reference ; remplace le provisionnel s'il existe

Transition : lorsqu'un document definitif arrive, le systeme :

1. Resout la serie cible (`seriesId`) via `supplier`, `label`, `extraFields` et `groupCols`.

2. Recherche une facture provisionnelle avec le meme `tenantId` , `pipelineId` , `seriesId` et la meme `date` .
3. Si trouvee, copie tous les champs de la definitive (montant, label, extraData, source, etc.) dans l'entite existante, passe `isFinal=true` , `accountingStatus="VALIDATED"` et `validationDate` / `lastValidatedDate` de la serie, met a jour les statistiques.
4. Sinon, cree une nouvelle facture definitive.

7.5 Saisonnalite automatique (Python)

Le script `compute_series.py` calcule pour chaque serie :

- Ecart type global (`std_global`) de tous les montants.
- Pour chaque mois ayant des donnees : ecart type de ce mois, puis moyenne de ces ecarts types mensuels (`avg_monthly_std`).
- Ratio = `avg_monthly_std / std_global` . Si ratio < 0,7 → `useSeasonality = true` , sinon `false` .
- Les mois sans aucune facture recoivent `monthlyMuMap[mois] = 0.0` .

7.6 Scoring et creation d'alertes

Le script `detect.py` recoit un lot de factures (avec leur serie) et calcule un score (0-100) pour chaque anomalie de montant (type `AMOUNT`) selon la formule :

FORMULE DE SCORING

```
excess = amount - max_acceptable
denominator = reference_mu * tolerance_pct / 100
score = min(100, 60 + (excess / denominator) * 25)

reference_mu = monthlyMuMap[mois] si useSeasonality=true, sinon mu
max_acceptable = reference_mu * (1 + tolerance_pct/100)
```

Seuil d'alerte = **60**. Si score > 60, une `Anomaly` et une `Alert` (status `PENDING`) sont creees.

7.7 Detection des documents manquants

Pour chaque serie active :

```
nextExpected = lastReceivedDate (ou lastDate) + median_gap_days
deadline = nextExpected + toleranceDays
Si today > deadline et qu'aucune alerte MISSING recente n'existe,
cree une anomalie (score=100, type MISSING) et une alerte.
```

7.8 Planification et rapports

Planification : via `schedule` dans `PipelineConfigDTO` : mode `CRON` (expression cron) ou `POLLING` (intervalle minutes). Le `PipelineSchedulerManager` utilise `TransactionTemplate` pour exécuter les tâches dans une transaction.

Rapports :

- `GET /pipelines/{id}/runs` — historique des exécutions
- `GET /admin/pipelines/runs/{runId}/logs` — détail des erreurs

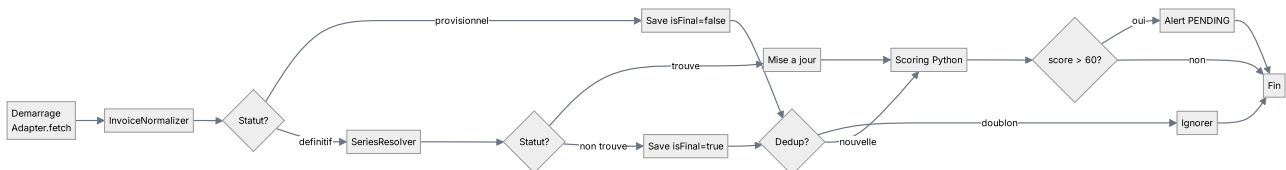


Figure 3 : Flux d'exécution d'un pipeline d'importation

8. Feature D — Connexion tenant ↔ ERP (authentification deleguee par jeton)

Ce n'est pas un SSO classique — l'ERP authentifie l'utilisateur localement, genere un jeton signe avec sa cle private, le transmet au moteur. Le moteur valide la signature avec la cle publique du connecteur et emet son propre jeton interne.

Logique metier

Un utilisateur deja connecte a son ERP (ex: commercial Whitecape) peut acceder au moteur via une iframe, sans nouveau mot de passe. L'ERP (via un module intermediaire) construit un "laissez-passer" numerique (jeton) contenant le nom de l'organisation et l'identifiant externe, le signe avec sa cle private, et l'envoie au moteur. Le moteur verifie la signature avec la cle publique du connecteur, puis s'assure que le tenant a bien cree une liaison pour cet identifiant. Si tout est valide, il delivre un jeton interne qui permet a l'utilisateur de naviguer dans l'interface.

Logique technique

Endpoint : `POST /auth/embed` (public, `permitAll`)

Corps :

```
{ "token": "external_jwt", "connectorName": "Ask&Go" }
```

Validation :

1. Recuperer le connecteur par `connectorName` → cle publique.
2. Valider la signature du JWT avec cette cle (support RSA et ECDSA).
3. Verifier le claim `exp` (expiration).
4. Extraire `tenant` (nom du tenant) et `external_id`.
5. Rechercher le tenant par son nom.
6. Verifier qu'une `TenantErpConnection` existe avec ce tenant, ce connecteur et cet `external_id`.

Succes : generer un jeton interne (JWT, expiration 1h) contenant `tenant_id`, `connector_id`, `external_id`, sans role Spring.

Filtrage des donnees : `TenantFilter` positionne `isErpSession=true` et les trois identifiants dans `TenantContext`. Tous les services utilisent alors des methodes repository qui ajoutent les conditions `connector_id = :connectorId` et `external_id = :externalId`.

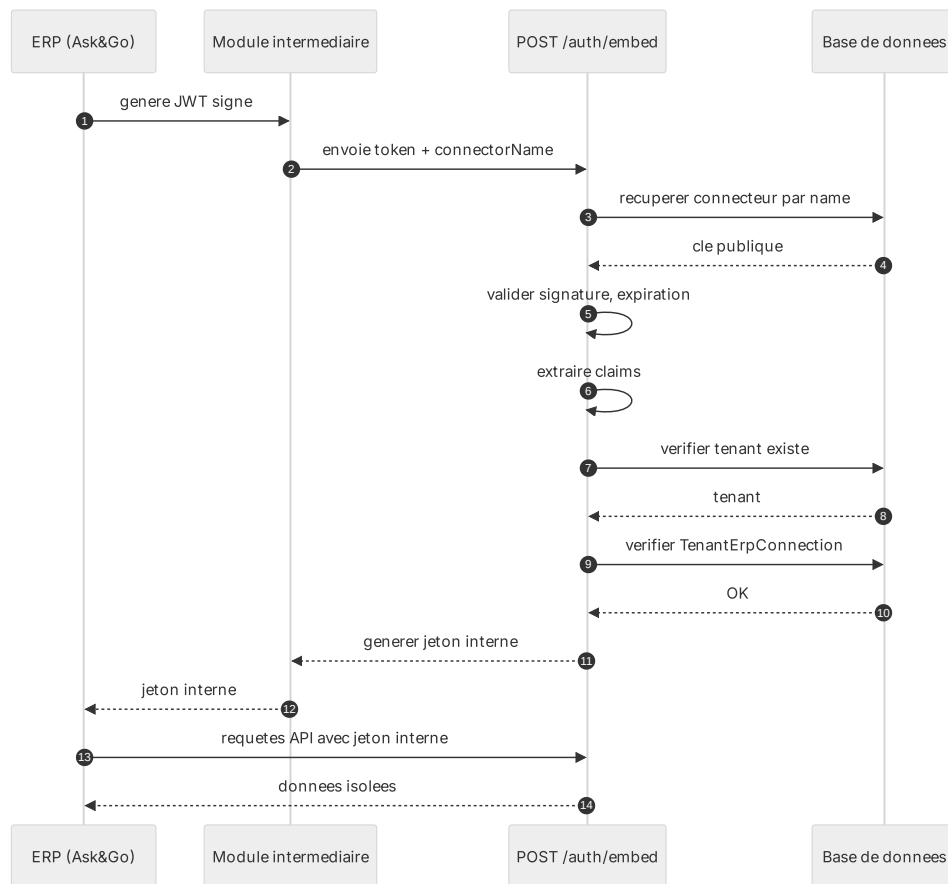


Figure 4 : Sequence d'authentification deleguee ERP

9. Feature E — Alertes et feedback

Logique metier

Lorsque le moteur detecte une anomalie (score > 60), il cree une alerte. L'utilisateur (administrateur tenant ou commercial via iframe) peut consulter l'alerte, voir l'explication, et choisir de la **Confirmer** (vraie anomalie), **Rejeter** (faux positif — le systeme ajuste sa tolerance), ou **Ignorer** (ne rien modifier). Toutes les decisions sont enregistrees.

Logique technique

Creation : `AlertService.createIfNeeded()` appele apres scoring.

Consultation : `GET /alerts?status=PENDING&type=AMOUNT&startDate=...&endDate=...` → page. Pour les sessions ERP, filtrage automatique par `connector_id` et `external_id`.

Historique : `GET /feedback/log` (filtre par pipeline) retourne les feedbacks avec anciennes/nouvelles tolerances.

Reponse : tableau d'alertes avec `score`, `severity`, `explanation`, etc.

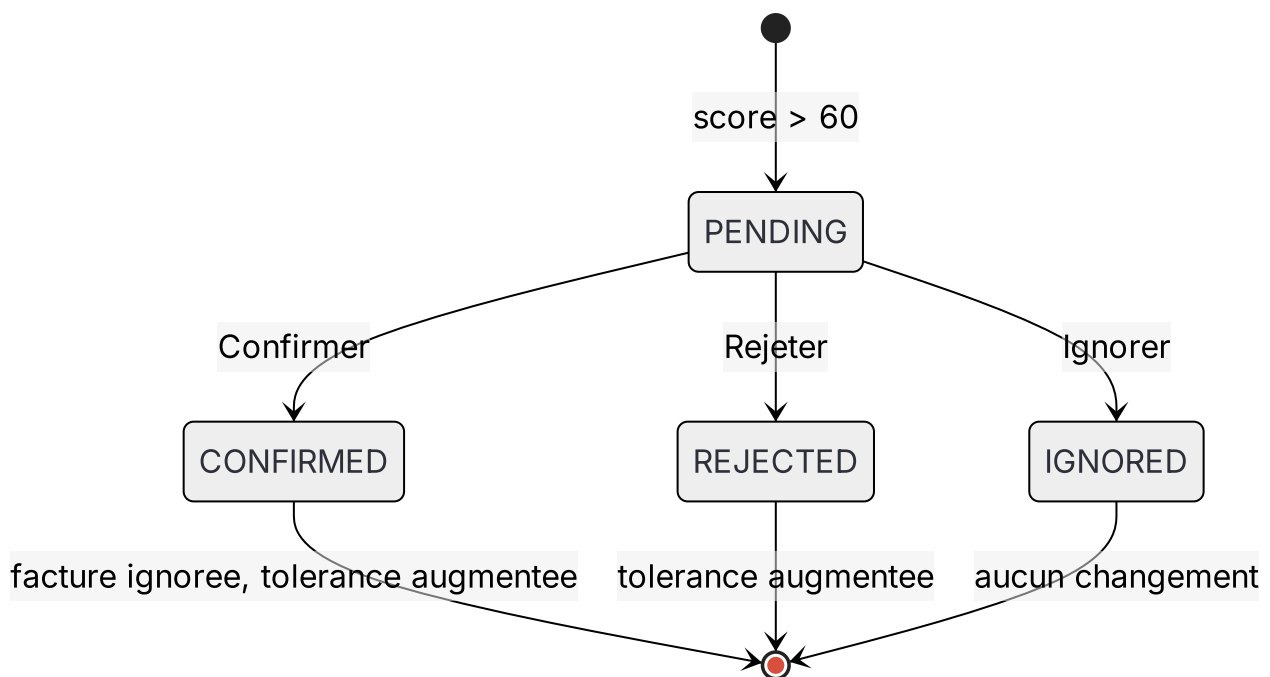


Figure 5 : Machine à états des alertes

Tableau 13 : Transitions des alertes

ACTION	STATUT RESULTAT	EFFET
Confirmer	CONFIRMED	Facture ignoree, tolerance augmentee
Rejeter	REJECTED	Tolerance augmentee

10. Feature F — Budget et previsions

Logique metier

Le moteur calcule automatiquement un budget annuel pour chaque serie (fournisseur + libelle) en analysant les depenses reelles des trois dernieres annees. Ce budget est ensuite reparti mois par mois en respectant le profil saisonnier detecte automatiquement (par exemple, plus de depenses en decembre, moins en aout). Vous pouvez tester d'autres montants via des simulations — mais rien n'est jamais sauvegarde en base de donnees. Les budgets sont toujours des projections calculees a la volee.

Logique technique

Le systeme budgetaire est un moteur de calcul en **lecture seule**. Aucun budget n'est persiste. Les seules ecritures en base liees aux budgets sont les alertes d'anomalie generees lors des scans de fond (`recalculate-all` , `refresh`).

10.1 Philosophie : aucune persistance

Tableau 14 : Comportement du systeme budgetaire

ASPECT	COMPORTEMENT
Budgets implicites	Calcules a la volee depuis l'historique des factures
Budgets simules	Projections ephemerres, jamais stockees
Alertes de simulation	de <code>AlertDTO</code> en memoire uniquement, jamais persistees
Alertes de fond	Persistees via <code>anomalyRepository.save()</code> lors des scans <code>recalculate-all</code> / <code>refresh</code>

La table `ai_budgets` existe dans le schema mais est inutilisee — conservee pour compatibilite ascendante.

10.2 Budget implicite (GET)

FORMULE

Budget Annuel Implicite = moyenne des sommes annuelles des annees N-1, N-2, N-3

- Seules les factures definitives (`isFinal = true`) sont prises en compte.
- Les annees a zero sont exclues du calcul de moyenne.
- Si aucun historique : budget = 0.

Repartition mensuelle :

Tableau 15 : Methodes de repartition mensuelle

CONDITION	METHODE
<code>useSeasonality = true</code>	Pattern depuis <code>monthlyMu</code> (map JSON mois → montant attendu historique)
<code>useSeasonality = false</code>	Repartition plate : <code>annualBudget / 12</code>

Scaling factor : dans les deux cas, un facteur d'echelle est applique pour que la somme des 12 montants mensuels egale *exactement* le budget annuel. La forme relative (saisonniere ou plate) est preservee.

Classification mensuelle :

Tableau 16 : Statuts mensuels du budget

STATUT	CONDITION
NORMAL	realAmount dans la tolerance
CRITICAL	Depassement > tolerancePct%
UNDER_CONSUMPTION	Sous-consommation > tolerancePct%
IN_PROGRESS	Mois en cours
UPCOMING	Mois futurs

Indicateurs complementaires :

- `confidence` : **HIGH** si ≥ 12 mois d'historique, **LOW** sinon.
- `estimatedOverrunMonth` : mois estime d'epuisement du budget (projection lineaire).

10.3 Simulation budgetaire (POST) — deux modes

Mode 1 : Serie unique

POST `/pipelines/{pipelineId}/series/{seriesId}/budget/simulate?annualBudget={montant}`

Comportement :

1. Charge la serie et son historique de factures.
2. Calcule `historicalAnnualAverage` (meme formule N-1/N-2/N-3 que l'implicite) — valeur informative uniquement.
3. Construit la repartition mensuelle avec le `annualBudget` fourni par l'utilisateur (meme logique saisonniere/uniforme + scaling).
4. Compare avec les realises de l'annee demandee.
5. Genere des alertes simulees (`BUDGET_MONTHLY_OVERRUN` , `BUDGET_ANNUAL_OVERRUN`) — en memoire uniquement.
6. Retourne `BudgetSimulationDTO` avec `budgetType = "SIMULATION"` .

Mode 2 : Global (toutes les series)

POST `/pipelines/{pipelineId}/budget/simulate?annualBudget={montant}&year={annee}`

Comportement :

1. Charge toutes les series actives (`active = true`) du pipeline.
2. Pour chaque serie, calcule son budget implicite historique (N-1/N-2/N-3).
3. Calcule `totalImplicitBudget` = somme de tous les budgets implicites.
4. **Allocation proportionnelle :**

$$\text{budgetAlloueSerie} = \text{annualBudgetUtilisateur} \times \left(\frac{\text{budgetImpliciteSerie}}{\text{totalImplicitBudget}} \right)$$

- Si `totalImplicitBudget = 0` : repartition egale (`annualBudget / nombreDeSeries`).
5. Pour chaque serie, appelle la meme logique de simulation que le mode unique avec son budget alloue.
 6. Retourne `List<BudgetSimulationDTO>` — un par serie.

CARACTERISTIQUES DE L'ALLOCATION

- **Respecte les patterns historiques** : une serie qui consomme 70% du budget historique recoit 70% du nouveau budget.
- **Profils saisonniers propres** : chaque serie conserve son propre profil saisonnier (`monthlyMu`).
- **Alertes independantes** : les alertes simulees sont independantes par serie.

10.4 DTOs de reponse

BudgetDTO (GET — budget implicite, format riche)

Tableau 17 : Champs du BudgetDTO

CHAMP	DESCRIPTION
<code>annualBudget</code>	Budget annuel calcule
<code>budgetType</code>	"IMPLICIT"
<code>months</code>	Tableau de 12 <code>MonthBudgetDTO</code> enrichis
<code>realYtd</code>	Realise annee en cours
<code>remainingBudget</code>	Reste a depenser
<code>projectedYearEnd</code>	Projection fin d'annee
<code>confidence</code>	HIGH / LOW
<code>estimatedOverrunMonth</code>	Mois d'epuisement estime

MonthBudgetDTO (riche) : `month`, `monthName`, `expectedAmount`, `realAmount`, `variance`, `variancePct`, `status`, `normalRhythmAmount`, `seasonalityAvgAmount`, `scalingFactor`, `priorYearsSpends`.

BudgetSimulationDTO (POST — simulation, format minimal)

Tableau 18 : Champs du BudgetSimulationDTO

CHAMP	DESCRIPTION
<code>seriesId</code>	UUID de la serie
<code>supplier</code> / <code>label</code>	Identifiants
<code>budgetType</code>	"SIMULATION"
<code>annualBudget</code>	Budget alloue (user input ou proportionnel)
<code>historicalAnnualAverage</code>	Moyenne historique N-1/N-2/N-3 (contexte)
<code>useSeasonality</code>	Flag saisonnalite
<code>monthlyBreakdown</code>	Map<Integer, MonthBudgetDTO> minimal
<code>triggeredAlerts</code>	List<AlertDTO> — alertes simulees en memoire

10.5 Alertes budgetaires — simulation vs persistance

Tableau 19 : Comparaison alertes simulees et persistantes

CONTEXTE	METHODE	PERSISTEE ?
Simulation (POST <code>/budget/simulate</code>)	<code>computeSimulationAlerts()</code>	Non — AlertDTO en memoire
Scan de fond (<code>recalculate-all</code> , <code>refresh</code>)	<code>checkAndGenerateAlerts()</code>	Oui — via <code>anomalyRepository.save()</code> + <code>AlertService.createIfNeeded()</code>

Parite de logique : `computeSimulationAlerts()` reproduit exactement la meme logique que `checkAndGenerateAlerts()` — memes seuils, memes severites, memes formats d'explication. La simulation predit fidelement quelles alertes seraient generees.

10.6 Endpoints

Tableau 20 : Endpoints du module budget

METHODE	ENDPOINT	DESCRIPTION	PERSISTANCE
GET	<code>/pipelines/{id}/series/{sid}/budget?year=Y</code>	Budget implicite (historique)	Non
POST	<code>/pipelines/{id}/series/{sid}/budget/simulate?annualBudget=</code>	Simulation serie unique	Non
POST	<code>/pipelines/{id}/budget/simulate?annualBudget=&year=</code>	Simulation globale (toutes series)	Non
POST	<code>/pipelines/{id}/series/{sid}/budget</code>	Ancien endpoint explicite — desormais simulation	Non
DELETE	<code>/pipelines/{id}/series/{sid}/budget?year=Y</code>	No-op (rien a supprimer)	—
POST	<code>/pipelines/{id}/series/budget/recalculate-all?year=Y</code>	Recalcul implicite + generation d'alertes persistees	Alertes uniquement
POST	<code>/pipelines/{id}/series/{sid}/budget/refresh</code>	Scan serie unique + alertes persistees	Alertes uniquement
GET	<code>/pipelines/{id}/series/{sid}/budget/automatic</code>	Detail du calcul moyenne (N-1/N-2/N-3)	Non
POST	<code>/pipelines/{id}/series/{sid}/budget/automatic/apply</code>	Identique au GET (lecture seule)	Non

10.7 Flux de donnees

Tableau 21 : Etapes du flux budgetaire

ETAPE	DESCRIPTION	SORTIE
1. Collecte	Factures definitives N-1, N-2, N-3	Somme par annee
2. Verification	Au moins une annee non nulle ?	Si non → Budget = 0
3. Calcul	Moyenne = Budget annuel implicite	BudgetDTO riche (GET)
4. Repartition	Mode saisonnier ou uniforme + scaling	12 montants mensuels
5. Allocation (mode global)	Budget alloue = $\text{userBudget} \times \text{part historique}$	BudgetSimulationDTO par serie
6. Comparaison	Realises vs attendus	Alertes simulees ou persistantes

10.8 Points cles de conception

1. **Idempotence** : meme entree → meme sortie, aucun effet de bord.
2. **Isolation simulation** : les endpoints `simulate` ne touchent aucun repository.
3. **Allocation proportionnelle** : le mode global respecte les poids historiques, pas une repartition egale arbitraire.
4. **Moteur unique** : `implicit` et `simulated` utilisent le meme `MonthlyPatternBuilder` (saisonnalite + scaling + tolerance).
5. **Parite alertes** : `computeSimulationAlerts()` $\equiv$ `checkAndGenerateAlerts()` — predictions fideles.
6. **Backward compatibility** : `ai_budgets` existe mais est inutilise ; ancien endpoint `POST /series/{sid}/budget` devient simulation sans persistance.

11. Feature G — Supervision et administration

Logique metier

L'administrateur du moteur dispose d'un tableau de bord avec des indicateurs globaux (nombre de tenants, pipelines actifs, alertes par statut, executions des 7 derniers jours, etc.) et peut afficher les memes indicateurs pour un tenant specifique. Il peut egalement lister toutes les liaisons tenant-ERP et en supprimer une pour nettoyer d'anciennes configurations.

Logique technique

GET `/admin/stats` → retourne `StatsDTO` : `tenantsCount` , `activePipelinesCount` , `totalPipelinesCount` , `executionsLast7Days` , `failedRunsLast7Days` , `successfulRunsLast7Days` , `invoicesCount` , `anomaliesCount` , `totalSeries` , `totalConnectors` , `alertsByStatus` .

GET `/admin/stats/tenant/{tenantId}` → memes indicateurs filtres par tenant.

GET `/admin/tenant-connections` → liste toutes les liaisons (tous tenants).

DELETE `/admin/tenant-connections/{id}` → supprime une liaison.

12. Sécurité et isolation des données

Tableau 22 : Mesures de sécurité

DOMAINE	IMPLEMENTATION
Isolation multi-tenant	Toutes les tables ont <code>tenant_id</code> ; <code>TenantFilter</code> injecte le <code>tenantId</code> dans <code>TenantContext</code> ; les repositories filtrent systématiquement.
Isolation intra-tenant	Les tables <code>ai_invoices</code> , <code>ai_series</code> , <code>ai_alerts</code> ont <code>connector_id</code> ; les sessions ERP filtrent sur ces colonnes.
Chiffrement	<code>AES/GCM/NoPadding</code> avec IV aléatoire, dérivation PBKDF2, pas de fallback en clair.
Mots de passe	BCrypt, complexité (majuscule, minuscule, chiffre, spécial).
Variables obligatoires	<code>ADMIN_PASSWORD</code> , <code>JWT_SECRET</code> , <code>ENCRYPTION_KEY</code> — démarrage impossible sans elles.
Injection SQL	Plus de <code>whereClause</code> libre ; conditions structurées + <code>PreparedStatement</code> .
Authentification déléguée	Validation de la signature, vérification de l'expiration, support RSA/ECDSA.

13. Exemples complets de flux API

13.1 Connexion administrateur tenant

```
curl -X POST https://moteur/api/auth/login \  
  -H "Content-Type: application/json" \  
  -d '{"username": "whitecape_admin", "password": "Secure123!"}'
```

Reponse :

```
{ "access_token": "...", "token_type": "Bearer", "expires_in": 3600 }
```

13.2 Creation d'un pipeline CSV

```
curl -X POST https://moteur/api/pipelines \  
  -H "Authorization: Bearer <token>" \  
  -H "Content-Type: application/json" \  
  -d '{"name": "Import EAU", "sourceType": "CSV"}'
```

13.3 Upload et preview CSV

```
curl -X POST https://moteur/api/pipelines/{id}/preview/csv \  
  -H "Authorization: Bearer <token>" \  
  -F "file=@EAU.csv"
```

Reponse preview :

```
{  
  "columns": ["supplier", "label", "amount", "date", "status"],  
  "sample": [{"supplier": "EAU_SAISON", "amount": "150.00", ...}]  
}
```

13.4 Configuration d'un pipeline JDBC avec conditions

```
{
  "jdbcUrl": "jdbc:postgresql://askandgo-db:5432/erp",
  "jdbcUsername": "readonly",
  "jdbcPassword": "***",
  "tables": [{"name": "factures", "alias": "f"}],
  "fieldMapping": {
    "supplier": "f.fournisseur_nom",
    "amount": "f.montant",
    "date": "f.date_facture"
  },
  "conditions": [
    { "column": "f.tenant_code", "operator": "=", "value": "whitecape_ask" },
    { "column": "f.montant", "operator": ">", "value": 0 }
  ]
}
```

13.5 Execution et historique

```
curl -X POST https://moteur/api/pipelines/{id}/run \
-H "Authorization: Bearer <token>"

curl -X GET https://moteur/api/pipelines/{id}/runs \
-H "Authorization: Bearer <token>"
```

Reponse run :

```
{ "id": "...", "status": "SUCCESS", "invoicesImported": 12, ... }
```

13.6 Consultation des alertes

```
curl -X GET "https://moteur/api/alerts?status=PENDING&type=AMOUNT&\
startDate=2024-01-01&endDate=2024-12-31" \
-H "Authorization: Bearer <token>"
```

Reponse : tableau d'alertes avec **score** , **severity** , **explanation** , etc.

14. Codes d'erreur et gestion des exceptions

Toutes les erreurs retournent un JSON structure :

```
{
  "status": 400,
  "error": "Bad Request",
  "message": "...",
  "timestamp": "2026-05-31T10:00:00Z"
}
```

Tableau 23 : Codes d'erreur HTTP

CODE	SIGNIFICATION	CAS TYPIQUES
400	Requete invalide	Champ manquant, mot de passe trop court
401	Non authentifie	Token manquant, expire, signature invalide
403	Interdit	Role insuffisant (ex: TENANT appelle <code>/admin/**</code>)
404	Ressource introuvable	Tenant, pipeline, serie non trouves
409	Conflit	Username deja utilise, <code>external_id</code> deja lie, connecteur utilise
500	Erreur interne	Exception non geree, probleme base de donnees

15. Annexes — tableaux récapitulatifs

15.1 Endpoints principaux (synthese)

Tableau 24 : Synthese de tous les endpoints

METHODE	ENDPOINT	ROLE	ROLE REQUIS
POST	/auth/login	Authentification	—
POST	/auth/embed	Echange jeton externe	—
POST	/admin/tenants	Creer un tenant	ADMIN
GET	/admin/tenants	Lister les tenants	ADMIN
DELETE	/admin/tenants/{id}	Supprimer un tenant	ADMIN
POST	/admin/connectors	Creer un connecteur	ADMIN
GET	/admin/connectors	Lister les connecteurs	ADMIN
DELETE	/admin/connectors/{id}	Supprimer un connecteur	ADMIN
POST	/pipelines	Creer un pipeline	TENANT
POST	/pipelines/{id}/preview	Tester une source	TENANT
PUT	/pipelines/{id}/mapping	Enregistrer la configuration	TENANT
POST	/pipelines/{id}/run	Executer le pipeline	TENANT
GET	/pipelines/{id}/runs	Historique des executions	TENANT
GET	/alerts	Lister les alertes	TENANT (ou ERP avec filtrage)
POST	/feedback/{alertId}	Traiter une alerte	TENANT (ou ERP)
GET	/pipelines/{id}/series/{id}/budget	Budget implicite	TENANT
POST	/pipelines/{id}/series/{id}/budget/simulate	Simulation	TENANT
GET	/admin/stats	Supervision globale	ADMIN
GET	/admin/tenant-connections	Lister liaisons (admin)	ADMIN

15.2 Variables d'environnement obligatoires

Tableau 25 : Variables d'environnement requises

VARIABLE	DESCRIPTION	EXEMPLE
ADMIN_PASSWORD	Mot de passe du compte admin	ADMIN_PASSWORD=MyV3ryS3cureP@ss
JWT_SECRET	Cle secrete JWT (≥32 caracteres)	JWT_SECRET=une-cle-tres-longue-et-aleatoire-32+
ENCRYPTION_KEY	Cle de chiffrement AES (≥16 caracteres)	ENCRYPTION_KEY=une-cle-16-car
SPRING_DATASOURCE_URL	URL de la base du moteur	—
SPRING_DATASOURCE_USERNAME	Utilisateur de la base	—
SPRING_DATASOURCE_PASSWORD	Mot de passe de la base	—

15.3 Proprietes configurables (application.yml)

```
detection:
  alert-score-threshold: 60
  severity:
    alerte-min: 61
    critique-min: 86

app:
  uploads:
    max-size: 104857600 # 100 MB
  cleanup:
    cron: "0 0 3 * * ?"

python:
  scripts:
    extract-dir: /tmp/anomaly-scripts
    executable: python3
    timeout-seconds: 60
```